

Department of Technology, SPPU

Project - High Level Design

On

Deepfake Face Detection

Using Deep Learning

Team members:

Vivek Deshmukh - Roll No:BT24DSL38

Industry Mentor:

Guided by:

Prof. Manisha Bharti

Table of Contents

1. Introduction.....	3
1.1. Scope of the document.....	3
1.2. Intended Audience.....	3
1.3. System overview.....	3
2. System Design.....	4
2.1. Application Design.....	4
2.2. Process Flow.....	5
2.3. Information Flow.....	6
2.4. Components Design.....	7
2.5. Key Design Considerations.....	8
2.6. API Catalogue.....	8
3. Data Design.....	9
3.1. Data Model.....	9
3.2. Data Access Mechanism.....	9
3.3. Data Retention Policies.....	10
3.4. Data Migration.....	10
4. Interfaces.....	11
5. State and Session Management.....	12
6. Non-Functional Requirements.....	13
6.1. Security Aspects.....	13
6.2. Performance Aspects.....	13
7. References.....	14

1.Introduction

1.1 Scope of the Document

This High Level Design document describes the overall system architecture, component interactions, data flow, and non-functional requirements for the Deepfake Face Detection project. The document covers the complete pipeline from dataset loading through model training, evaluation, and deployment via the Gradio web application.

1.2 Intended Audience

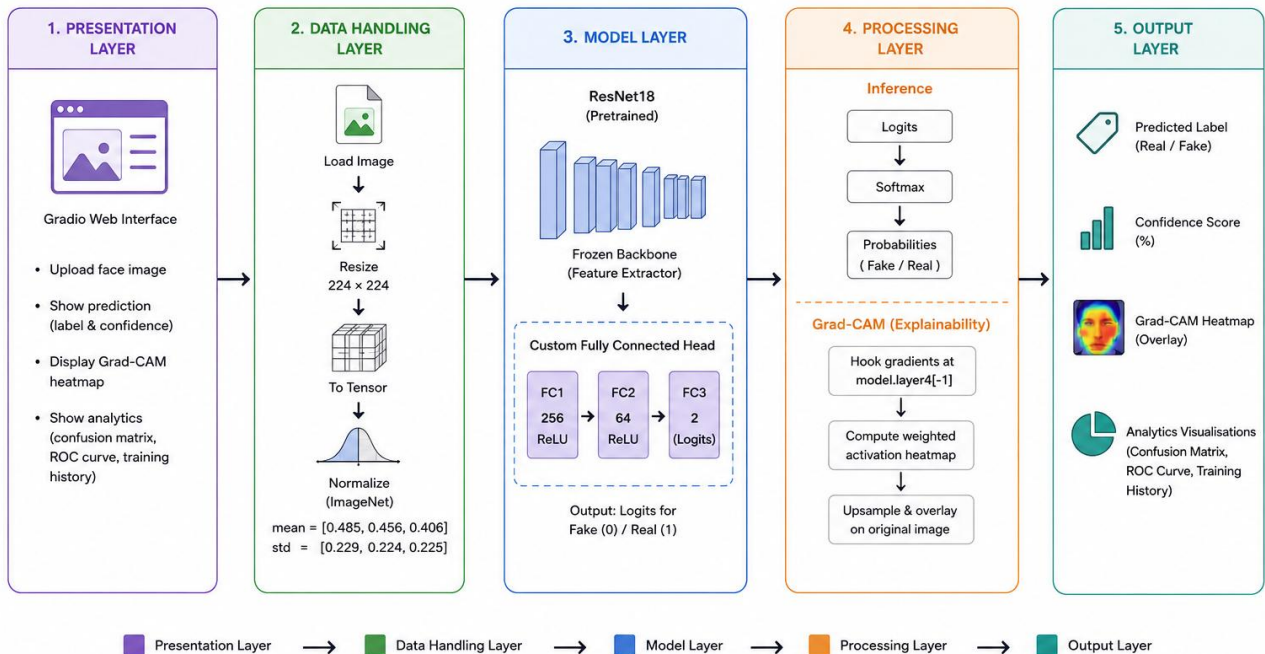
This document is intended for the IBM industry mentor, academic guide, and technical reviewers evaluating the project. It assumes familiarity with machine learning concepts, Python-based deep learning frameworks, and web-based deployment tools.

1.3 System Overview

The system is a binary image classifier that determines whether a given face image is real or AI-generated (fake). It is built using a ResNet18 convolutional neural network with a pretrained ImageNet backbone and a custom three-layer classification head. The model is trained on the 140k Real and Fake Faces dataset containing 100,000 training, 20,000 validation, and 20,000 test images. Predictions are served through a Gradio web interface that displays Grad-CAM explainability overlays and a descriptive analytics dashboard showing training curves, confusion matrix, and ROC curve.

2. System Design

2.1 Application Design



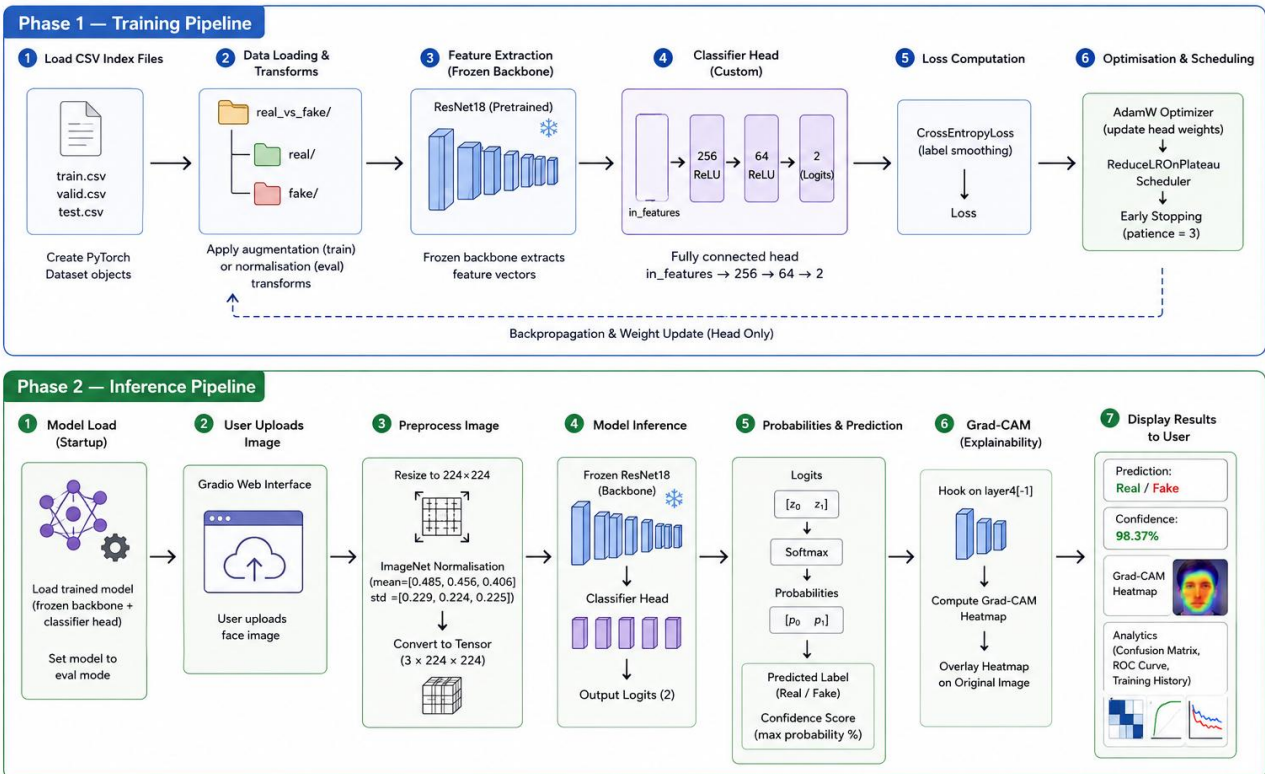
The system is designed using a modular architecture to ensure efficient and scalable performance. The pipeline flows from user input through preprocessing, model inference, post-processing, and final output as shown in the architecture diagram above.

- Presentation Layer:**
 Handles all user interaction via the Gradio web interface. Accepts uploaded face images, displays prediction results with confidence scores, renders the Grad-CAM heatmap overlay, and shows analytics charts (confusion matrix, ROC curve, training history).
- Data Handling Layer:**
 Manages input image loading and preprocessing. Resizes images to 224×224 pixels, converts to tensors, and applies ImageNet normalisation (mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]) to prepare inputs compatible with the pretrained ResNet18 backbone.
- Model Layer:**
 Contains the trained ResNet18-based classifier. The frozen pretrained backbone extracts deep convolutional features, which are passed to a custom fully connected head (in_features → 256 → 64 → 2) that outputs logits for the fake and real classes.
- Processing Layer:**
 Core inference layer where model logits are converted to class probabilities via Softmax. Also drives the Grad-CAM module — hooking gradients at model.layer4[-1] to compute a weighted activation heatmap highlighting the facial regions most influential to the prediction.

• Output Layer:

Packages and returns the final results to the presentation layer: the predicted label (Real / Fake), the confidence percentage, the Grad-CAM heatmap overlaid on the original image, and pre-rendered analytics visualisations from saved artifact files.

2.2 Process Flow



The end-to-end process flow of the system is described below in two phases:

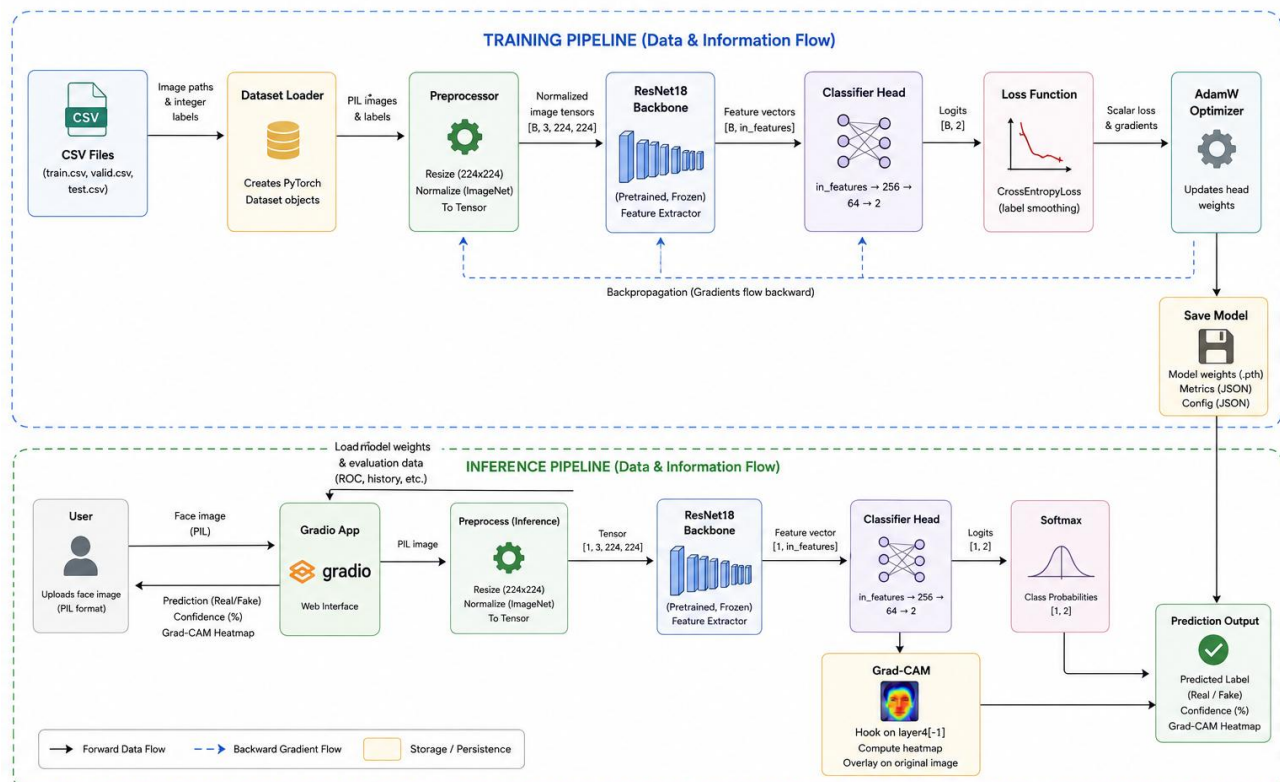
Phase 1 — Training Pipeline:

- CSV index files (train.csv, valid.csv, test.csv) are loaded to construct PyTorch Dataset objects
- Images are fetched from the real_vs_fake/ directory and passed through augmentation (train) or normalisation (eval) transforms
- Preprocessed image tensors are fed into the frozen ResNet18 backbone to extract feature vectors
- Feature vectors are passed to the custom classifier head (in_features → 256 → 64 → 2) to produce logits
- CrossEntropyLoss with label smoothing is computed; AdamW optimiser updates the head weights
- ReduceLROnPlateau scheduler adjusts the learning rate; early stopping halts training after patience=3 epochs without improvement

Phase 2 — Inference Pipeline:

- Gradio app loads model at startup and sets the model to eval mode
- User uploads a face image via the browser interface
- Image is preprocessed (resize to 224×224, ImageNet normalisation) and converted to a tensor
- Tensor is forwarded through the full model (frozen backbone + classifier head)
- Softmax converts logits to class probabilities; the higher probability class is returned as the prediction
- Grad-CAM hooks on layer4[-1] compute a heatmap overlay on the original image
- Prediction label, confidence score, and Grad-CAM heatmap are displayed to the user

2.3 Information Flow



The following describes how information moves between the key components of the system:

- **CSV Files → Dataset Loader:**
Transfers image paths and corresponding integer labels.
- **Dataset Loader → Preprocessor:**
Passes raw PIL images along with labels.
- **Preprocessor → ResNet18 Backbone:**
Sends normalized image tensors of shape [B, 3, 224, 224].

- **ResNet18 Backbone → Classifier Head:**
Transfers extracted feature vectors [B, in_features].
- **Classifier Head → Loss Function:**
Outputs logits of shape [B, 2].
- **Loss Function → AdamW Optimizer:**
Provides scalar loss value and computed gradients.
- **Training Pipeline → Save Model:**
Stores trained model weights, metrics (JSON), and configuration settings.
- **Save Model → Gradio App:**
Loads trained model weights and evaluation data (ROC, training history).
- **User → Gradio App:**
Uploads face image (PIL format) through the interface.
- **Gradio App → User:**
Returns prediction (Real/Fake), confidence percentage, and Grad-CAM heatmap.

2.4 Components Design

The system is designed using modular components to ensure efficient and maintainable processing:

- **Dataset Loader:**
Reads image paths and labels from CSV index files and returns PyTorch Dataset objects.
Handles label encoding (fake=0, real=1).
- **Preprocessor Component:**
Applies training augmentations (random crop, horizontal flip, colour jitter) and evaluation-time normalisation using ImageNet mean and std values.
- **ResNet18 Backbone:**
Pretrained ImageNet feature extractor with all weights frozen. Provides deep convolutional feature representations fed into the custom head.
- **Classifier Head:**
Custom fully connected layers (in_features → 256 → 64 → 2) with ReLU activations and Dropout(0.3). Only this component is trained.
- **Training Pipeline:**
Manages the training loop, validation, AdamW optimiser, ReduceLROnPlateau scheduler, early stopping, and checkpoint saving.
- **Grad-CAM Module:**
Registers gradient hooks on model.layer4[-1] to produce visual heatmaps explaining each prediction.
- **Gradio Application:**
Browser-based deployment interface exposing image upload, prediction output, Grad-CAM visualisation, and an analytics dashboard.
- **Analytics Notebook:**
Renders confusion matrix, ROC curve, per-class metrics, and training history for reporting and presentation.

2.5 Key Design Considerations

- Frozen backbone strategy: keeping ResNet18 weights frozen reduces training cost significantly and prevents overfitting, while still leveraging powerful ImageNet features
- Balanced dataset: equal class distribution (50% real, 50% fake) across all splits eliminates the need for class-weighted loss or oversampling
- Transfer learning: using a pretrained backbone avoids training from scratch on 140k images, making the project feasible on consumer-grade GPU hardware
- Grad-CAM interpretability: visual explanations help validate that the model focuses on facial features rather than dataset-specific artefacts or background patterns
- Label smoothing: reduces overconfidence in predictions and improves generalisation by preventing the model from assigning extreme probability values

2.6 API Catalogue

This project does not expose a REST API. All interactions are handled through internal Python functions and the Gradio interface:

- **predict(image):**
Accepts a PIL image and returns prediction label, confidence score, and Grad-CAM heatmap.
- **generate_gradcam(image, model):**
Hooks into layer4[-1] of the model to generate a heatmap overlaid on the input image.
- **get_transform(mode):**
Returns data transformations — augmentation for training or normalization for evaluation/testing.
- **train_one_epoch(model, loader, ...):**
Executes one training epoch and returns average loss and accuracy.
- **evaluate(model, loader, ...):**
Performs model evaluation and returns loss, accuracy, F1-score, ROC-AUC, and confusion matrix.
- **load_checkpoint(path):**
Loads trained model weights (best_model.pt) into the model during application startup.

3.Data Design

3.1 Data Model

The system uses a file-system-based data model and does not rely on a relational database. The key data entities are as follows:

- **Face Images:**
Stored in JPEG format (~140,000 files). Images are resized to 256px and organized into *real/* and *fake/* folders under *train/*, *valid/*, and *test/* directories.
- **CSV Index Files:**
Stored as CSV files . Contain image paths and labels, with approximately 100k rows for training and 20k each for validation and testing.
- **Model Checkpoint:**
Stored as a .pt file . Contains the PyTorch state dictionary of the best-performing model (*best_model.pt*).
- **Training History:**
Stored as a JSON file. Includes epoch-wise training and validation metrics such as loss, accuracy, F1-score, and ROC-AUC.
- **Metrics Files:**
Stored as JSON files . Includes *best_valid_metrics.json* and *test_metrics.json* containing final evaluation results.
- **ROC Curve Data:**
Stored as JSON files. Contains False Positive Rate (FPR) and True Positive Rate (TPR) values for plotting ROC curves.
- **Configuration File:**
Stored as a JSON file. Contains complete training hyperparameters and system configuration (*config.json*).

3.2 Data Access Mechanism

- Image data is accessed via file path strings stored in CSV index files; PIL.Image.open() is used to load images on demand during training and inference
- CSV files are read using pandas at the start of each training run to construct the dataset index
- Model checkpoint is loaded via torch.load() once at Gradio app startup and held in memory for the duration of the session
- JSON artifact files are read via Python's built-in json module at app startup and notebook execution time
- No database queries, ORM, or network file access is used; all data resides on the local file system

3.3 Data Retention Policies

- Model artifacts (best_model.pt, metrics JSONs) are overwritten each training run when a new best validation checkpoint is found
- Raw dataset images are read-only and are never modified or deleted by the system
- User-uploaded images in the Gradio app are held in memory only during inference and are not written to disk
- No user data is logged, stored, or transmitted externally by the system

3.4 Data Migration

- The dataset is downloaded once from Kaggle and stored locally in the real_vs_fake/ directory; no automated migration pipeline is implemented
- CSV index files (train.csv, valid.csv, test.csv) are provided with the dataset and require no transformation before use
- To migrate to a different dataset, the CSV files must be recreated with the same schema (path, label columns) and image files placed in the expected directory structure

4. Interfaces

The Deepfake Face Detection System includes multiple interfaces that enable interaction between users, system components, and internal modules. These interfaces ensure smooth data flow, usability, and effective system operation.

1. User Interface (UI)

The User Interface provides a platform for users to interact with the system through the Gradio web application. It allows users to upload face images and view prediction results in real-time. The interface displays the predicted label (Real/Fake), confidence score, Grad-CAM heatmap, and analytics charts. It is designed to be simple, responsive, and user-friendly.

2. Input Interface

The Input Interface handles image acquisition from the user. It accepts face images (JPEG/PNG) uploaded via the browser and ensures they are correctly passed to the preprocessing module in the required format (PIL image).

3. Model Interface

The Model Interface connects the application with the deep learning model (ResNet18-based classifier). It manages the transfer of preprocessed image tensors to the model and retrieves output logits, which are further processed into predictions and probabilities.

4. Processing Interface

The Processing Interface performs intermediate operations such as applying Softmax to convert logits into probabilities and generating Grad-CAM visualizations. It ensures that model outputs are transformed into interpretable results.

5. Output Interface

The Output Interface presents the final results to the user. It displays the predicted class (Real/Fake), confidence percentage, and Grad-CAM heatmap overlaid on the input image. It also shows analytics such as confusion matrix, ROC curve, and training history.

6. Storage Interface

This interface manages interaction with stored data such as model checkpoints, metrics, and configuration files. It handles loading the trained model and accessing saved evaluation results required for inference and visualization.

5.State and Session Management

The system is designed to be largely stateless to keep deployment simple:

- **Gradio Application State:**
The model checkpoint and analytics data are loaded once at app startup and held in memory as global Python objects for the lifetime of the process. Each inference request is independent and stateless — no user session data is maintained between requests.
- **Training State:**
The training pipeline maintains epoch-level state (current best validation loss, early stopping counter, learning rate) in memory during the training run only. This state is not persisted between runs; a new training run always starts fresh.
- **Model Eval Mode:**
The model is explicitly set to eval() mode before every inference call to disable dropout and batch normalisation training behaviour, ensuring deterministic and consistent predictions.
- **No User Sessions:**
The Gradio app does not implement user sessions, authentication tokens, or cookies. It is designed for single-user local deployment and does not track any user-specific state.

6. Non-Functional Requirements

6.1 Security Aspects

- **Input Validation:**
The Gradio interface accepts only valid image formats (JPEG, PNG). Invalid or unsupported files are rejected at the UI level.
- **No Data Persistence:**
Uploaded images are processed in memory only during inference and are not stored on disk or shared externally.
- **Local Deployment:**
The application runs on localhost (127.0.0.1), restricting access to the local machine and minimizing network exposure.
- **Limited Access Control:**
The current setup is intended for single-user academic use and does not include authentication. For production use, security enhancements such as HTTPS, authentication, and rate limiting would be required.
- **Secure Dependencies:**
The application relies on widely used and trusted libraries (PyTorch, Gradio, NumPy, Pillow), reducing the risk of vulnerabilities from unverified sources.

6.2 Performance Aspects

- **Inference Latency:**
The application provides near real-time predictions. On GPU, inference (including Grad-CAM) typically completes in under 1 second, while on CPU it takes approximately 2–5 seconds.
- **Resource Utilization:**
The deployed model is lightweight (~45 MB) and requires less than 1 GB GPU memory during inference, making it suitable for local systems.
- **Startup Time:**
The application initializes quickly by loading the trained model and required resources at launch, with startup time typically under 10 seconds.
- **Responsiveness:**
The interface processes one image at a time and returns results promptly, ensuring a smooth user experience.
- **Scalability:**
The current implementation supports single-user interaction. For multi-user scenarios, deployment on a production server with request handling would be necessary.
- **Consistency:**
The model provides stable and consistent predictions for the same input under identical conditions.

7.Reference

- [1] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. IEEE CVPR 2016.
- [2] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. IEEE ICCV 2017.
- [3] Karras, T., Laine, S., & Aila, T. (2019). A Style-Based Generator Architecture for Generative Adversarial Networks. IEEE CVPR 2019.
- [4] Rossler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner, M. (2019). FaceForensics++: Learning to Detect Manipulated Facial Images. IEEE ICCV 2019.
- [5] Tolosana, R., Vera-Rodriguez, R., Fierrez, J., Morales, A., & Ortega-Garcia, J. (2020). Deepfakes and Beyond: A Survey of Face Manipulation and Fake Detection. Information Fusion, 64, 131–148.
- [6] PyTorch Documentation. <https://pytorch.org/docs/stable/index.html>
- [7] Gradio Documentation. <https://www.gradio.app/docs>
- [8] Dataset: xhlulu. (2020). 140k Real and Fake Faces. Kaggle. <https://www.kaggle.com/datasets/xhlulu/140k-real-and-fake-faces>